

Observability with LangSmith

Adding end-to-end tracing to the LangGraph chatbot — how LangSmith captures every turn, organizes traces into threads, and why observability matters as agentic systems grow more complex.

WHAT THIS CHAPTER COVERS

- ▶ What observability means for LLM/agentic systems
- ▶ Creating a LangSmith account and API key
- ▶ Environment variables that enable automatic tracing
- ▶ How a trace, run, and project relate to each other
- ▶ The thread-fragmentation problem in LangSmith
- ▶ Solving it with metadata and run_name
- ▶ Updating `backend.py` / `frontend.py` config for tracing
- ▶ Why this matters for future tools/RAG/MCP chapters

Observability with LangSmith

RECAP — CHAPTER 14

The previous chapter added a **SQLite checkpointer** to the chatbot, giving it database persistence — conversations survive across restarts because LangGraph's state is checkpointed per *thread_id*. The chatbot at this point already has a Streamlit GUI, response streaming, and persistent multi-conversation storage. This chapter adds the next capability on top of that same codebase: **observability**.

1. What Is Observability, and Why Does It Matter?

Definition

Observability, in the context of an LLM/agentic application, means being able to **trace the end-to-end execution** of every user interaction — capturing exactly what message the user sent, what the system internally did with it, what the model returned, and metrics such as token usage and latency — all recorded automatically in a dedicated tool.

Why It Exists

As soon as a chatbot is more than a single prompt-response pair, it becomes difficult to answer basic questions just by looking at the UI: What exactly was sent to the LLM? How many tokens did that turn cost? How long did the model take to respond? Which node in the LangGraph graph executed? Observability tooling answers these questions without requiring any manual logging code.

Problem It Solves

Without observability, debugging or auditing an agentic system means adding scattered `print()` statements or custom logging, which becomes unmanageable once the graph grows (multiple nodes, tool calls, retries, sub-agents). Observability tooling centralizes this information in one dashboard, per conversation turn, without touching the application's core logic.

The Tool: LangSmith

This chapter uses **LangSmith**, the observability platform built by the LangChain team, to trace the LangGraph chatbot built across the previous chapters. LangSmith automatically records:

- The exact input and output of every node execution
- Which LLM was called and what it returned
- Token usage (input tokens and output tokens)
- Latency, including time-to-first-token
- Execution start time, end time, and status

REAL WORLD INSIGHT

Once agentic systems add tools, retrieval (RAG), or MCP integrations, tracing becomes far more valuable — it lets an engineer see exactly which tool was called, with what arguments, and what it returned, without instrumenting the code by hand. Setting up observability early, on a simple chatbot, builds the habit before complexity arrives.

🔄 Quick Revision — Observability

- Observability means tracing a system's end-to-end execution automatically.
- It answers: what was sent to the LLM, what came back, how many tokens, how long it took.
- LangSmith is the observability/tracing tool used for this chatbot.
- No manual logging code is required once LangSmith is configured — tracing happens automatically in the background.

2. Setting Up LangSmith

Step 1 — Create an Account and API Key

1. Go to **smith.langchain.com** and sign in (or create an account).
2. Open **Settings** → **API Keys**.
3. Click **Create API Key**, provide a short description, and confirm.
4. Copy the generated API key somewhere safe — it is shown only once.

Step 2 — Add Environment Variables

LangSmith is enabled purely through environment variables — no changes to the LangGraph application code are required to turn tracing on. Add the following to the project's `.env` file:

```
# .env
LANGSMITH_TRACING=true
LANGSMITH_ENDPOINT=<YOUR_LANGSMITH_ENDPOINT_URL>
LANGSMITH_API_KEY=<YOUR_API_KEY>
LANGSMITH_PROJECT=chatbot project
```

IMPORTANT

The `LANGSMITH_API_KEY` value must be exactly the key generated in Step 1. Never commit this key to source control or paste it into shared notes — treat it like any other secret credential.

1. WHAT EACH VARIABLE DOES

Variable	Purpose
----------	---------

LANGSMITH_TRACING	Set to <code>true</code> to turn tracing on for the project
LANGSMITH_ENDPOINT	The LangSmith API URL the traces are sent to
LANGSMITH_API_KEY	Authenticates the application with the LangSmith account
LANGSMITH_PROJECT	The project name under which traces are grouped in the dashboard

BEST PRACTICE

Give `LANGSMITH_PROJECT` a meaningful, application-specific name (e.g. `chatbot project`). LangSmith automatically creates this project the first time the application runs — it does not need to be created manually on the dashboard.

🔄 Quick Revision — Setup

- An API key is generated once, from Settings → API Keys, on smith.langchain.com.
- Four environment variables enable tracing: `LANGSMITH_TRACING`, `LANGSMITH_ENDPOINT`, `LANGSMITH_API_KEY`, `LANGSMITH_PROJECT`.
- No application code changes are needed just to turn tracing on.
- The named project is auto-created in LangSmith the first time the app runs with these variables set.

3. Automatic Tracing — How It Works

How It Works

Once the environment variables are set, LangSmith instruments LangChain/LangGraph's internals automatically. Every time the compiled graph executes, LangSmith silently records what happened — there is no need to wrap function calls in tracing decorators or emit manual log events.

Re-running the exact same `backend.py` / `frontend.py` code from Chapter 14 (the SQLite-persisted chatbot), with only the `.env` file updated, is sufficient to start seeing traces appear on the LangSmith dashboard.

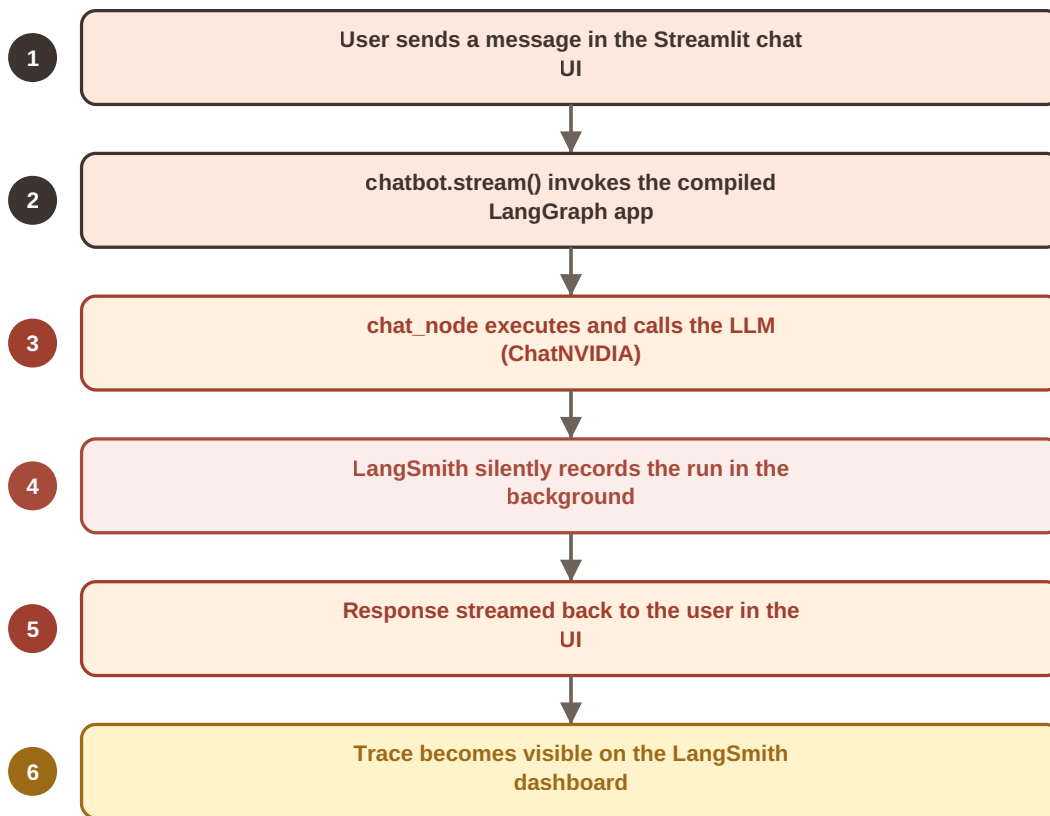


Figure 15.1 — A user turn is automatically captured as a LangSmith trace with no code changes to the chatbot itself.

COMMON MISTAKE

Expecting to see traces without restarting the application after editing `.env`. Environment variables are read once at process startup — the chatbot process must be restarted for new LangSmith variables to take effect.

🔄 Quick Revision — Automatic Tracing

- Tracing is fully automatic once the environment variables are set — zero application code changes required.
- The graph's normal behavior (streaming, responses) is completely unaffected by tracing.
- The application process must be (re)started after changing `.env` for the new variables to apply.

4. The Trace Hierarchy: Project → Trace → Run

Definition

LangSmith organizes captured data in three levels:

- **Project** — the top-level container, named via `LANGSMITH_PROJECT` (e.g. *chatbot project*). All traces from this application land here.

- **Trace** — a single end-to-end execution of the graph. In this chatbot, one trace corresponds to **one conversational turn**: the user sends one message and the graph produces one reply.
- **Run** — the individual step(s) inside a trace, such as the `chat_node` executing and calling the LLM. A simple one-node graph produces one run per trace; a more complex graph with multiple nodes/tool calls would show several runs nested inside a single trace.

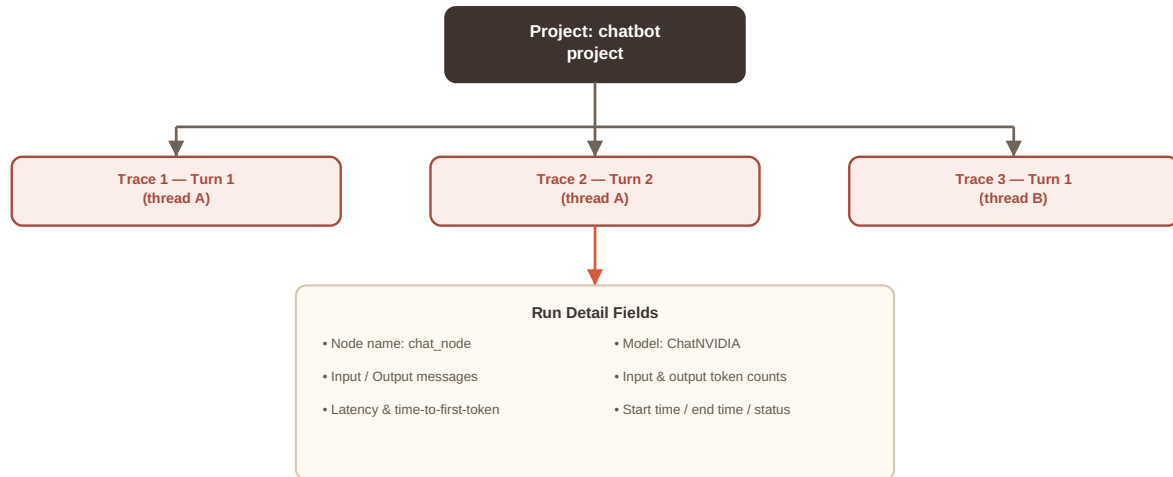


Figure 15.2 — A LangSmith project contains one trace per conversational turn; each trace records detailed run-level execution data.

What a Run Records

Clicking into any trace shows, at minimum:

- The node name that executed (e.g. `chat_node`) and the model used (e.g. ChatNVIDIA)
- The exact input and output messages
- Input token count and output token count
- Latency and time-to-first-token
- Execution start time, end time, and status

For example, in the lecture demo, one turn was the user asking “Give me a road map to study AI Engineering” — this single question-and-answer pair appeared as one trace, with the full input/output and timing metrics visible inside it.

INTERVIEW TIP

If asked to explain LangSmith's data model, describe it top-down: **Project** (the whole application) → **Trace** (one end-to-end execution, i.e. one conversational turn here) → **Run** (an individual step/node execution inside that trace). This mirrors how distributed tracing works in general software observability (e.g. spans within a trace).

🔄 Quick Revision — Trace Hierarchy

- Project is the top-level container, named by `LANGSMITH_PROJECT`.
- A trace = one full execution of the graph, i.e. one conversational turn.
- A run = an individual node/step execution inside a trace (e.g. `chat_node` calling the LLM).
- Each run/trace records input, output, tokens, latency, and status automatically.

5. The Thread-Fragmentation Problem

Problem It Solves

By default, LangSmith simply lists every trace flatly inside the project, in the order they occurred. If a user starts **multiple separate conversations** (multiple threads) with the chatbot, their traces from different conversations end up interleaved in the same flat list — there is no way, out of the box, to tell which traces belong to which conversation.

This mirrors a distinction the chatbot's own persistence layer already understands: LangGraph groups checkpointed messages by `thread_id` (see Chapter 14). LangSmith, however, does not automatically know about this `thread_id` unless it is explicitly told.

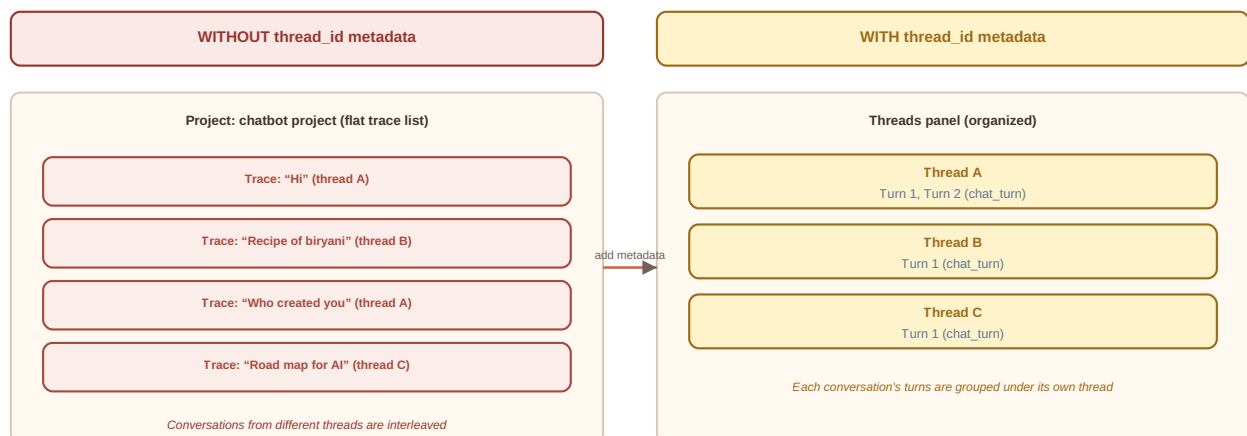


Figure 15.3 — Without explicit thread metadata, traces from different conversations are interleaved; with it, LangSmith groups every turn under its own thread.

IMPORTANT

Passing `thread_id` inside `configurable` (as already done for the SQLite checkpointer) only controls LangGraph's own state persistence. It does **not**, by itself, group traces into LangSmith's Threads view — that requires a separate, explicit step, covered next.

🔄 Quick Revision — Thread Fragmentation

- By default, LangSmith traces from all conversations appear flatly in one project.
- This makes it hard to review a single conversation's history from the dashboard.
- The configurable.thread_id used for LangGraph persistence does not automatically group LangSmith traces.
- LangSmith exposes a separate 'Threads' feature to solve this.

6. Solving It: Grouping Traces into Threads

How It Works

LangSmith supports organizing traces into **Threads**, but this requires explicitly attaching a `thread_id` (or `session_id` / `conversation_id`) to the run's **metadata** at invocation time. This is done via the same `config` dictionary already being passed to `chatbot.stream()` — it just needs two additional keys.

1. METADATA.THREAD_ID — GROUPS TRACES INTO A THREAD

Adding a `metadata` key to the config, containing the same `thread_id` already used for checkpointing, tells LangSmith which thread each trace belongs to.

2. RUN_NAME — IMPROVES READABILITY

By default, every trace is named `LangGraph`, which is not descriptive. Setting `run_name` to something meaningful (e.g. `chat_turn`) makes each trace immediately recognizable in the dashboard as representing one turn of conversation.

BEFORE — CONFIG ONLY

```
CONFIG = {
    'configurable': {
        'thread_id':
    st.session_state['thread_id']
    }
}
```

AFTER — WITH METADATA + RUN NAME

```
CONFIG = {
    'configurable': {
        'thread_id':
    st.session_state['thread_id']
    },
    'metadata': {
        'thread_id':
    st.session_state['thread_id']
    },
    'run_name': 'chat_turn'
}
```

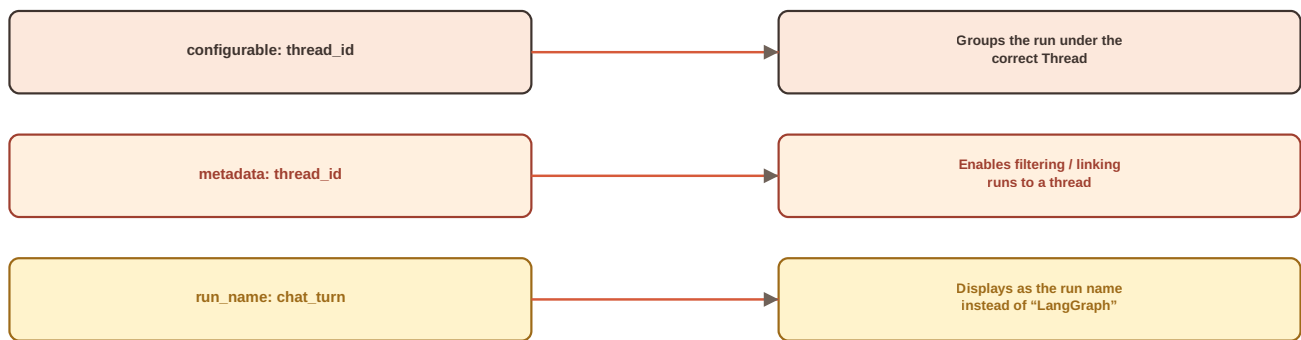


Figure 15.4 — Each field added to `CONFIG` maps directly to a specific behavior inside the LangSmith dashboard.

BEST PRACTICE

Reuse the exact same `thread_id` value inside both `configurable` (for LangGraph checkpointing) and `metadata` (for LangSmith thread grouping). Keeping a single source of truth for the thread identifier avoids the two systems ever disagreeing about which turn belongs to which conversation.

COMMON MISTAKE

`run_name` is optional and purely cosmetic — skipping it does not break thread grouping. Confusing it with the required `metadata.thread_id` key is a common mix-up; only the `metadata` key is required to populate the Threads view.

🔄 Quick Revision — Grouping Traces

- Add a `metadata` key to `CONFIG` containing the same `thread_id` used for checkpointing.
- This makes the trace appear under its corresponding conversation in LangSmith's Threads panel.
- `run_name` (e.g. 'chat_turn') is an optional, cosmetic label replacing the default 'LangGraph' name.
- The same `thread_id` value should be reused across `configurable` and `metadata` for consistency.

7. Implementation Walkthrough — `frontend.py`

Where the Change Lives

The only file that changes for this chapter is `frontend.py` — specifically, the `CONFIG` dictionary built right before `chatbot.stream()` is called. `backend.py` (the graph definition, state, and SQLite checkpointer from Chapter 14) is untouched.

```
# frontend.py — inside the "if user_input:" block
CONFIG = {
    'configurable': {'thread_id': st.session_state['thread_id']},
    'metadata': {
        'thread_id' : st.session_state['thread_id']
    },
    'run_name': 'chat_turn'
}

# Stream the AI response — unchanged from Chapter 14
with st.chat_message('assistant'):
    ai_message = st.write_stream(
        message_chunk.content for message_chunk, metadata in chatbot.stream(
            {'messages': [HumanMessage(content = user_input)]},
            config = CONFIG,
            stream_mode = 'messages'
        )
    )
```

Note that `chatbot.stream(..., config=CONFIG, ...)` itself is unchanged — only the shape of the `CONFIG` dictionary passed into it grew two new keys. This is the essence of why observability integrates so cleanly: the tracing behavior is attached through configuration, not through rewriting execution logic.

INTERVIEW TIP

If asked how LangSmith integrates with an existing LangGraph app with minimal disruption, the answer is: (1) environment variables turn tracing on globally, and (2) an optional `metadata/run_name` addition to the existing `config` dict organizes traces into threads — no change to node functions, graph structure, or the checkpointing is required.

🔄 Quick Revision — Implementation

- Only `frontend.py`'s `CONFIG` dictionary changes; `backend.py` stays exactly as in Chapter 14.
- `chatbot.stream()` itself is called identically to before — only `CONFIG`'s contents change.
- Tracing plus thread grouping is achieved purely through configuration, not new logic.

K. Key Takeaways

- **Observability** means automatically tracing a system's end-to-end execution — inputs, outputs, tokens, and latency — without manual logging code.
- **LangSmith** is enabled purely through four environment variables (`LANGSMITH_TRACING`, `LANGSMITH_ENDPOINT`, `LANGSMITH_API_KEY`, `LANGSMITH_PROJECT`) and requires zero application code changes to start tracing.

- LangSmith organizes data as **Project** → **Trace** → **Run**; in this chatbot, one trace equals one conversational turn.
- By default, traces from all conversations are listed flatly in one project, making it hard to follow a single conversation — the **thread-fragmentation problem**.
- Adding `thread_id` to the config's **metadata** key (in addition to `configurable`) groups traces into LangSmith's **Threads** view, matching the application's own conversation structure.
- **run_name** is an optional cosmetic label that replaces the default "LangGraph" trace name with something descriptive, like "chat_turn".
- The only code change required is inside `frontend.py`'s `CONFIG` dictionary — `backend.py` and the graph itself remain unchanged.

R. Revision Sheet

Observability

Automatic end-to-end tracing of a system's execution — inputs, outputs, tokens, latency — without manual logging.

Run

An individual node/step execution inside a trace, e.g. `chat_node` calling the LLM.

LangSmith

The observability platform used to trace this LangGraph chatbot; enabled via environment variables only.

Thread Fragmentation

The default problem where traces from different conversations appear interleaved in one flat project list.

Project

The top-level LangSmith container for an application, named by `LANGSMITH_PROJECT`; auto-created on first run.

`metadata.thread_id`

The config key that groups traces into LangSmith's Threads view, using the same `thread_id` as checkpointing.

Trace

One full execution of the graph — here, one conversational turn (user message + reply).

`run_name`

An optional config key that gives each trace a descriptive name (e.g. "chat_turn") instead of the default "LangGraph".

I. Interview Questions

Q1. What is observability in the context of an LLM/agent application, and why does it matter more as systems grow more complex?

Q2. How is LangSmith tracing enabled for a LangGraph application, and does it require any changes to the graph's node functions?

Q3. Explain LangSmith's data hierarchy: how do Project, Trace, and Run relate to one another?

Q4. In this chatbot, what does a single "trace" correspond to?

Q5. What information does a single run capture inside a LangSmith trace?

Q6. What is the thread-fragmentation problem in LangSmith, and why does it occur by default?

Q7. Does passing `thread_id` inside `configurable` (for LangGraph checkpointing) automatically group traces into LangSmith Threads? Why or why not?

Q8. What config key needs to be added, and with what value, to group a chatbot's traces by conversation in LangSmith?

Q9. What is the purpose of the `run_name` config key, and is it required for thread grouping to work?

Q10. Which file(s) in the chatbot's codebase needed to change to add LangSmith observability, and which stayed the same?

F. Further Reading

- LangSmith official documentation (smith.langchain.com) — tracing, projects, and threads concepts
- LangSmith "Log your first thread" guide — the exact metadata pattern used in this chapter
- LangGraph documentation — on config, checkpointers, and `thread_id`
- LangSmith observability crash-course video referenced by the instructor (linked in the original lecture description)